SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: Drone Route Planning in a Flood Zone (Greedy Best-First Search vs A*)

Problem set-up: States are locations (grid cells / waypoints); actions are moves between them. g(n) is the accumulated travel cost (terrain + weather) from the base, and h(n) is the straight-line distance to the destination. Blocked paths are edges that disappear or become infinitely costly.

i. Greedy Best-First Search (GBFS) in this scenario

GBFS always expands the node with the smallest h(n) (f(n) = h(n)). The drone therefore flies "as the crow flies" towards the target, keeping only a small frontier.

- **Strengths:** very fast, few node expansions and low memory, which suits an urgent delivery; simple to re-run whenever new information arrives.
- **Weaknesses under uncertainty:** it ignores g(n), so it does not notice that a short leg crosses heavy wind or difficult terrain; it is neither optimal nor (in graphs with dead ends) guaranteed complete; a blocked road or flooded valley in front of the goal traps it in a local minimum of h; it commits confidently to routes based on stale partial updates, which is risky; worst-case time and space are O(b^m).

ii. A* in the same situation

A* expands the node with the lowest **f(n) = g(n) + h(n)**. g(n) captures what has actually been paid (variable costs from terrain and weather), while h(n) pulls the search towards the goal. Because straight-line distance never overestimates the true cost (it is admissible, and consistent), A* is **complete and optimal**. When a path is reported blocked, that edge is removed (cost = ∞) and the remaining frontier nodes still hold alternative routes, so A* naturally switches to the next best one. Its drawback is more expansions and O(b^d) memory, which can be reduced with Weighted A* (f = g + w·h) or incremental versions such as D* Lite.

iii. Impact of heuristic accuracy

Heuristic quality	Greedy Best-First	A*
Accurate (close to true cost)	Heads almost straight to the goal; very fast and often good.	Few expansions; optimal path found quickly.
Weak / loose (e.g. ignores rivers, weather)	Led into dead ends and detours; path can be very poor.	Still optimal if admissible, but expands many more nodes (h = 0 degenerates to uniform-cost search).
Overestimating (inadmissible)	No guarantee in any case; may look fast but wrong.	Optimality is lost; may skip the best route.

Here the straight-line estimate is admissible but optimistic, because floods force detours. A weather-aware heuristic (distance × minimum cost factor) tightens it without breaking admissibility.

iv. Effect of dynamic changes (blocked paths)

- A blocked path invalidates the current plan and forces **re-planning**; the search effort rises with each change and delay grows.
- GBFS is cheap to re-run but can oscillate or loop between choices because it has no memory of cost.
- A* re-planned from scratch is expensive; **incremental search (D* Lite / LPA*)** reuses earlier work and repairs only the affected part. With partial information, the drone assumes unknown cells are passable and re-plans when its sensors reveal a block.

v. Recommendation

Use A* (preferably Weighted A* with incremental re-planning, e.g. D* Lite). Real-time: a weight w ≈ 1.2–1.5 or an anytime variant bounds the delay while keeping the path within a known factor of optimal. Optimality: with an admissible heuristic the route minimises total cost. Safety: risk (flood depth, wind, no-fly zones) can be added into g(n), so safer routes are preferred over merely shorter ones, something GBFS cannot do. GBFS is kept only as an emergency fallback if the time budget is exhausted.

Question 2: Smart-City Traffic Signal Optimisation (Hill Climbing, Simulated Annealing, Genetic Algorithms)

Formulation as an optimisation problem

- **Decision variables:** x = (cycle length, green split, offset) for each of the N intersections.
- **Objective:** minimise f(x) = w_1 ·(total waiting time) + w_2 ·(fuel consumption) + w_3 ·(congestion index).
- **Constraints:** minimum/maximum green times, pedestrian crossing times, bounded cycle lengths, no conflicting greens.

Why traditional search is inefficient: the space grows as k^N (k settings per intersection, hundreds of intersections), so BFS/DFS/A* cannot enumerate it; there is no single goal state or useful path, only a best configuration; every evaluation needs a traffic simulation or live data; and the traffic pattern keeps changing, so a stored solution goes stale.

i. Hill Climbing and its limitations

Start from the current timing plan. Neighbours are produced by small changes (e.g. ±5 s green at one junction or a shift in offset). Evaluate f, move to the best improving neighbour (steepest-ascent) and repeat until no neighbour is better. It uses O(1) memory, is fast and works online. **Limitations:** it stops at local optima, stalls on plateaus, is slowed by ridges, depends on the starting plan, never backtracks, and is misled by noisy traffic measurements.

ii. Local maxima, plateaus and ridges (real-world meaning)

- **Local maximum:** every single-junction change makes the network worse, so the system settles on a "good" plan, e.g. a long green on the main road helps that junction, but the city-wide best needs a coordinated "green wave" that no single change reveals.
- **Plateau:** a flat region where changing a green phase by a few seconds does not change measured waiting time (e.g. saturated roads or very light night traffic), so there is no direction to climb.
- **Ridge:** the improvement lies along a narrow path needing several junctions to change together (shifting offsets of A, B and C at once); each single move goes downhill, so hill climbing zig-zags or halts.

iii. How Simulated Annealing and Genetic Algorithms overcome these limitations

- **Simulated Annealing:** also accepts a worse neighbour with probability e^(−Δ/T). Early on, with high temperature T, it explores widely (in simulation, not on the road); as T cools it behaves like hill climbing. This lets it escape local maxima and cross plateaus, and with slow cooling it approaches the global optimum.
- **Genetic Algorithms:** keep a population of timing plans (chromosome = timings of all intersections), with fitness = −f(x). Selection keeps good plans, **crossover** combines good corridors from different plans (handling ridges and coordination), and **mutation** injects new variation. Diversity prevents the whole search from collapsing into one local peak, and evaluation parallelises well.

iv. Recommended approach

A hybrid (memetic) strategy: a Genetic Algorithm (with SA for local refinement) computes good baseline plans per traffic pattern (morning peak, evening peak, events), and a lightweight online Hill-Climbing/low-temperature SA fine-tunes the live plan. Scalability: the city can be split into zones optimised in parallel, and GA scales with computing power. Adaptability: the online stage is warm-started from the current best plan, so it reacts to changing traffic in seconds, while re-running the GA periodically avoids drift into poor local optima.

Question 3: Autonomous Mars Rover (Online Search)

i. Why an online search agent is needed

An **offline** agent computes a complete plan before acting and so needs a full, correct model of the world. The rover has no complete map, can only learn what an action leads to by trying it, and meets hazards (craters, rocks) only as its sensors reach them. Earth–Mars communication delay (minutes) rules out remote driving. An **online** agent interleaves planning and acting: choose an action, execute, observe, update and repeat, which is the only workable approach in unknown terrain.

ii. Challenges of partial observability and dynamic environments

- **Partial observability:** short, noisy sensor range, hidden craters beyond view or under dust, ambiguous percepts, and localisation drift, so the exact state is uncertain and the agent maintains a belief.
- **Dynamic environment:** dust storms, changing light and shadows, loose soil that shifts, and changing battery/thermal state mean earlier knowledge may become wrong.
- **Safety and cost:** some actions are irreversible (falling into a crater), exploration consumes limited energy and time, and bad decisions cannot be undone, so the terrain must be "safely explorable".

iii. How the rover builds and updates its internal model

- Keep an **occupancy/cost map**: each cell is unknown, free, obstacle or hazard, with a probability; everything starts as unknown.
- At every step: **sense** → **localise** (odometry + visual SLAM) → **update cells** (Bayesian/log-odds update) → update traversal costs and heuristic estimates H(s) → **re-plan** with the new map.
- Store a result table (state, action) → next state as in LRTA*, a visited set to avoid loops, and the locations of hazards and sample sites.

iv. Online search compared with uninformed and informed search

Aspect	Uninformed (BFS, DFS, UCS)	Informed (Greedy, A*)	Online (LRTA*, online DFS, D* Lite)
Map needed	Full map and offline	Full map and heuristic, offline	No full map; uses local percepts
Guidance	None; blind expansion	Heuristic towards goal	Heuristic that is learned/updated while moving
Handles unknowns	No	No (plan breaks when map is wrong)	Yes; re-plans as knowledge grows
Cost of backtracking	Not modelled (simulated)	Not modelled (simulated)	Real, physical movement, so it is measured by competitive ratio
Completeness	Yes on known finite graphs	Yes with admissible h on known graphs	Yes in finite, safely explorable spaces

v. Most suitable approach

An online, informed, incremental search such as D* Lite / LRTA*, with hazard-aware costs. Uncertainty: the rover keeps a belief map and treats low-confidence cells cautiously. Adaptability: incremental re-planning reuses earlier computation, so new obstacles trigger only local repair. Safety: hazard cells receive very high or infinite cost, moves are restricted to reversible/safe states, safety margins are widened when sensor confidence is low, a retreat path to a known-safe area is always kept, and a safe-mode stop is the fallback. The heuristic is the estimated distance to the nearest sample site.

Question 4: University Exam Timetabling as a Constraint Satisfaction Problem

i. Variables, domains and constraints

- **Variables:** for every course c, X_c = exam time slot, H_c = exam hall (and optionally F_c = invigilator/faculty).
- **Domains:** D(X_c) = {Day1-Morning, Day1-Afternoon, …, DayK-Afternoon}; D(H_c) = halls with capacity ≥ number of students in c; D(F_c) = faculty qualified to invigilate/evaluate c.

Constraints:

Requirement	Constraint form	Type
No student has overlapping exams	$X_a \neq X_b$ for every pair of courses a, b that share a student	Binary
Limited exam halls	(X_c, H_c) pairs must all differ; number of exams in a slot ≤ number of halls; capacity(H_c) ≥ size(c)	Global / unary
Some subjects before others	$X_a < X_b$ for each prerequisite order a → b	Binary
Faculty availability	$X_c \in$ available slots of F_c; no faculty has two exams in one slot	Unary / binary
Special accommodations	Restricted domains: extra-time slots, separate or accessible halls	Unary

Taken together this is a graph-colouring-style CSP: courses are nodes, an edge joins courses that share students, and colours are time slots.

ii. Backtracking search

Backtracking assigns one variable at a time (depth-first), checks the constraints against variables already assigned, and **backs up** to the most recent choice as soon as a violation or empty domain appears. For example: assign DBMS → Day1-AM; AI (shares students with DBMS) cannot use Day1-AM, so it gets Day1-PM; if a later course ends with no legal slot, the search returns to the previous course and tries its next value. Worst case it is $O(d^n)$, so it is combined with heuristics: **MRV** (pick the variable with the fewest legal values), **degree heuristic** (most constraints), and **LCV** (try the value that rules out the fewest options).

iii. How constraint propagation (forward checking) improves efficiency

After each assignment X_c = v, **forward checking** deletes v from the domains of all neighbouring (constrained) courses and from hall/faculty domains made unavailable. If any domain becomes empty, the search backtracks immediately instead of discovering the dead end many levels later. Stronger propagation such as **arc consistency (AC-3 / MAC)** also prunes values that have no supporting value in a neighbour. Result: smaller branching, far fewer backtracks and earlier failure detection, at a small per-step cost that is normally repaid many times over.

iv. Real-world challenges and the best strategy

- **Scale:** thousands of courses and students give dense constraint graphs and large domains (timetabling is NP-hard).
- **Over-constrained instances:** there may be no perfect solution, so soft constraints (spread exams, limit three per day) are needed alongside hard ones.
- **Change:** withdrawals, hall closures, or faculty changes demand quick repair rather than a full re-solve.

Recommended strategy: model the hard constraints as a CSP and solve with backtracking + MRV/degree/LCV + forward checking/MAC, decomposing the problem (by department or connected components of the constraint graph) to keep sub-problems small. For large instances and for late changes, apply **min-conflicts local search** starting from the current timetable, and minimise a weighted penalty for soft constraints. This combines the completeness of systematic search on the hard core with the scalability and quick repair of local search.

Question 5: Strategic Game AI (Minimax and Alpha-Beta Pruning)

i. How Minimax models the problem

The game is treated as a **two-player zero-sum game** (the AI is **MAX**, the human is **MIN**). The game tree has game states as nodes and legal moves as edges; leaf (or depth-limited) states get a utility from an evaluation function. Values back up the tree: a MAX node takes the **maximum** of its children, a MIN node the **minimum**. The AI chooses the root move with the best guaranteed value, assuming the opponent replies optimally. Because the game is real-time, time is discretised into decision steps, units/actions are abstracted into macro-actions, and hidden information (fog of war) is handled by assumed or sampled states.

ii. Computational challenges of large game trees

- Time is O(b^d) and space O(b·d); with a branching factor b ≈ 30 and depth d = 8 there are roughly 6.5×10^{11} nodes, impossible within a 50–100 ms decision window.
- Each node needs a costly evaluation of a complex state; depth limits cause the **horizon effect**; real-time play, simultaneous moves, randomness and imperfect information add further difficulty.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta keeps **α** (the best value MAX can already guarantee on the current path) and **β** (the best value MIN can guarantee). Whenever **α ≥ β** the remaining children of that node are skipped.

- **Reasoning:** at a MIN node, once a child ≤ α is found, MIN can hold MAX to at most that value, which is no better than what MAX already has elsewhere, so MAX will never choose this branch and its other children cannot change the root decision. A symmetric argument prunes MAX nodes against β.
- **Same answer, less work:** the result is identical to full Minimax. With perfect move ordering the cost falls from O(b^d) to **O(b^(d/2))**, so the AI can search about twice as deep in the same time; with random ordering it is about O(b^(3d/4)). Ordering (captures/threats first, killer and history heuristics, best move from the previous iteration) and a **transposition table** to avoid re-searching repeated states bring it closer to the best case.

iv. Evaluation function and justification

Use a weighted linear function, $\text{Eval}(s) = w_1 \cdot \text{Material} + w_2 \cdot \text{Economy} + w_3 \cdot \text{Army strength/composition} + w_4 \cdot \text{Map control} + w_5 \cdot \text{Technology} + w_6 \cdot \text{Base safety} + w_7 \cdot \text{Mobility/initiative}$, where terminal wins/losses are ±∞.

Factor	Why it is included
Material (units and their value)	Direct measure of fighting capacity; the largest single predictor of winning.
Economy (resources, income, workers)	Long-term capacity to rebuild and out-produce the opponent.
Army strength and composition	Counters matter (e.g. ranged vs. melee); not only the number of units.
Map control / territory	Gives vision, expansion options and denies the enemy resources.
Technology and upgrades	Potential strength that does not yet appear in the unit count.
Base safety (health, exposure)	Losing the base ends the game, so threats to it carry a heavy penalty.
Mobility / initiative	Freedom of action and tempo; avoids being forced into reactive play.

Weights are normalised and tuned using self-play or machine learning (e.g. TD learning), and the function is kept cheap (near O(1) per state) because it is called at every leaf. A quiescence search in volatile positions (ongoing battles) reduces horizon-effect errors.

v. Strategy balancing optimality and real-time performance

Depth-limited Alpha-Beta with iterative deepening: search depth 1, 2, 3, … until the time budget expires and play the best move from the last completed depth (an anytime algorithm). Add move ordering, transposition tables, quiescence search, action abstraction (macro-actions),

selective/forward pruning and parallel evaluation. Use an opening book and cached evaluations for the early game, and adapt the depth to the time left or the difficulty level. Slight loss of theoretical optimality is accepted in exchange for guaranteed, responsive decisions; for extremely wide or hidden-information situations, Monte-Carlo Tree Search can complement it.